the correct order. Multicolumn indexes are also useful in other scenarios, but only the first column for queries that work with that column.

For a query with column "b" that above Index won't be used. What if your application needs an index to efficiently filter on column "b"? PostgreSQL recommends creating separate indexes for each of column "a" and "b".

The documentation page *Combining Multiple Indexes*[4] demonstrates how multiple Indexes can be scanned for a query, or even multiple scans of the same Index can be used to combine Index data for AND and OR conditions.

For the example above, having single column indexes on "a" and "b" and a query that filters on A and B, would allow PostgreSQL to combine the results of Index scans on each of the single column indexes.

Since Indexes add some latency to all DML (Inserts, Updates, and Deletes) operations, it's worth avoiding redundant indexes. This implies that having all three indexes mentioned so far, a single column index covering "a", a single column Index covering "b", and a Multicolumn index covering "a" and "b" is redundant.

On the other hand, a Multicolumn index that covers two columns where the first column is for filtering and the second column is for ordering, may provide the best query performance. You may wish to have specific redundancy like this when your Multicolumn index is specialized for certain queries, and single columns indexes are both used for others.

What types of Indexes are best for columns with boolean values?

Indexing Boolean Columns

Indexing columns that store boolean values has some pitfalls. The column may have three possible values when nulls are allowed, otherwise it will have only two possible values.

Imagine a column called `is_deleted` that's false by default, does not allow nulls, and is set to `TRUE` when the row is soft deleted. The majority of the rows will have the column value of false.

Adding an index to the `is_deleted` column may not be helpful depending on the queries. When querying for false values, which are the majority of the rows, PostgreSQL will likely scan the whole table with a Sequential Scan to filter out rows with true values that are Soft Deleted rows.

4. https://www.postgresql.org/docs/current/indexes-bitmap-scans.html